

DESVENDANDO O HASH: JENKINS ONE-AT-A-TIME

Hagatta Martins

TABELAS HASH

É uma estrutura de dados que armazena informações associando uma chave a um índice gerado por uma função de hash, permitindo buscas, inserções e remoções rápidas.

Principais características:

- Reduz o tempo de busca
- Organiza dados de forma eficiente
- Utiliza uma função de hash para mapear as chaves

JENKINS ONE AT A TIME HASH

O Jenkins One At A Time Hash é um metodo de algoritmo simples e eficiente, desenvolvido por Bob Jenkins, muito usado para strings de tamanho pequeno a médio.

Passo a passo do algoritmo

1. Para cada caractere da chave:

- Some o valor do caractere ao hash
- Aplique um shift à esquerda de 10 bits
- Faça uma operação XOR com o hash deslocado 6 bits para a direita

ENTENDENDO NA PRÁTICA

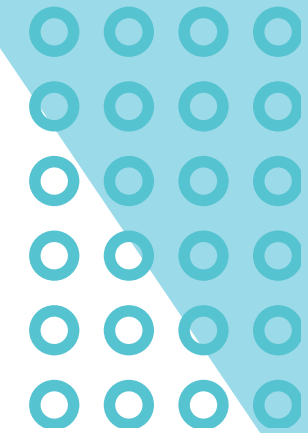
- Passo a Passo com a chave "Ana"

1. Começamos com 'A':

- O algoritmo pega o valor da letra 'A'.
- Ele "mistura" este valor com as operações matemáticas.
- Resultado: Temos um primeiro número de hash, que representa a informação do 'A'.

2. Adicionamos o 'N':

- O algoritmo pega o valor da letra 'n' e combina-o com o resultado anterior.
- Em seguida, mistura tudo de novo. Agora, as informações do 'A' e do 'n' estão completamente embaralhadas. O efeito do 'A' continua presente, mas foi modificado pelo 'n'.
- Resultado: O nosso número de hash foi atualizado e está mais complexo.



ENTENDENDO NA PRÁTICA

3. Finalizamos com o segundo 'A':

- O processo repete. O valor do último 'a' é adicionado ao resultado acumulado.
- Tudo é misturado uma última vez no loop principal. Cada letra influenciou o resultado final, criando uma "impressão digital" numérica única para a palavra "Ana".

4. O Resultado (Índice da Tabela):

- Após a etapa de finalização (as três últimas misturas), temos um número final. Para saber em que "gaveta" da tabela guardar "Ana", usamos o resto da divisão:
 - $\text{Índice} = \text{Número_Final} \% \text{Tamanho_da_Tabela}$

CÓDIGO DA FUNÇÃO DE HASH (JENKINS EM C)

```
// Função de Hash
static int jenkins_one_at_a_time_hash(const char *key) {
    int hash = 0; //inicializa em 0, não é modificada pela função
    while (*key) {
        hash += *key++; // pega o valor do caractere e soma a hash
        hash += (hash << 10); // multiplica a hash por 2^10 e soma a hash
        hash ^= (hash >> 6); // faz um XOR com a hash deslocada 6 bits para a direita e soma a hash
    }
    // Emabarralhamento final
    hash += (hash << 3); // multiplica a hash por 2^3 e soma a hash
    hash ^= (hash >> 11); // faz um XOR com a hash deslocada 11 bits para a direita e soma a hash
    hash += (hash << 15); // multiplica a hash por 2^15 e soma a hash
    return hash % TABLE_SIZE; // divide hasg por tamanho da tabela e retorna o resto
}
```

DESEMPENHO

```
--- Conteudo da Tabela Hash ---  
[0]: NULL  
[1]: Leo -> Julia -> NULL  
[2]: NULL  
[3]: Bia -> Ana -> NULL  
[4]: NULL  
[5]: Zoe -> NULL  
[6]: Batma -> Anderson -> NULL  
[7]: Coringa -> Lucas -> NULL  
[8]: Nosredna -> Hagatta -> NULL  
[9]: NULL  
-----
```

Lista Utilizada

```
--- Estatisticas de Desempenho ---  
Total de Elementos na Tabela: 11  
Total de Colisoes na Insercao: 5  
Taxa de Colisao: 50.00%  
  
Total de Buscas Realizadas: 4  
Tempo Medio por Busca: 0.000000 ms  
-----
```

Resultado gerado

RECOMENDAÇÃO

SITUAÇÕES DE USO RECOMENDADAS

- Verificador Ortográfico em Tempo Real;
- Detecção de Itens Duplicados;
- Contador de Frequência de Palavras (Análise de Texto)

SITUAÇÕES DE USO NÃO RECOMENDADAS

- Aplicações de Segurança;
- Grandes Volumes de Dados;
- Sistemas Críticos;



VANTAGENS

- Simplicidade: Fácil de implementar e entender.
- Velocidade: É extremamente rápido, ideal para o loop interno de uma tabela hash.
- Boa Distribuição: Oferece uma excelente distribuição para uso geral em tabelas hash, com poucas colisões inesperadas.

DESVANTAGENS

- NÃO é Criptográfico: Este algoritmo nunca deve ser usado para fins de segurança (como guardar senhas ou assinaturas digitais). É trivial criar colisões de propósito (duas chaves diferentes que geram o mesmo hash).
- Existem Alternativas Modernas: Para aplicações que exigem o máximo desempenho, algoritmos mais recentes como MurmurHash ou xxHash podem oferecer resultados ainda melhores.

CONCLUSÃO

O algoritmo Jenkins One-at-a-Time é um exemplo brilhante de engenharia de software: uma solução simples, elegante e eficaz para um problema complexo.

- **Processamento Sequencial:** Aborda a chave de entrada de forma metódica, um byte de cada vez, garantindo que nenhuma parte da informação seja ignorada.
- **Mistura Sofisticada de Bits:** Utiliza uma série inteligente de operações de bit (adição, shifts e XOR) para dobrar e espalhar a informação, criando um hash robusto.
- **Efeito Avalanche Pronunciado:** Garante que a mais pequena alteração na entrada provoque uma grande mudança na saída, resultando numa distribuição exceccionalmente uniforme dos dados.

LINK REPOSITÓRIO

<https://github.com/HttaMartins/Jenkins-One-At-A-Time-Hash>



OBRIGADA!!